CS3160 Lab 2

Functions and the RISC-V Calling Convention

What you will do today

You will write three functions — a simple one, a recursive one, and one with three arguments — and in doing so learn the calling convention that lets pieces of code that were written separately to call each other (and work seamlessly).

Last week your program was one lump of instructions and you could put anything anywhere. This week your functions are called by *our* test code, which has never seen your source. It will pass arguments where the convention says arguments go, and expect the answer to come from your code where the convention says answers go — and it will check that you left registers it was using alone.

Your folder is collected at the end of the session, and **only what is in it at that moment is graded**. You may keep working afterwards for your own practice.

1 Register names

Lab 1 had you write `x5`, `x11` and so on, because nothing had yet given those registers meaning. Now they have meaning, so use the ABI names from here on: `a0` for the first argument, `ra` for the return address, `sp` for the stack pointer. They are still the same registers — `x10` *is* `a0` — but the name now tells you what it is for.

Register	Name	Role	Who preserves it
`x1`	`ra`	return address	caller
`x2`	`sp`	stack pointer	**callee**
`x5-x7`	`t0-t2`	temporary	caller
`x8-x9`	`s0-s1`	saved	**callee**
`x10-x11`	`a0-a1`	arguments 1–2, return value	caller
`x12-x17`	`a2-a7`	arguments 3–8	caller
`x18-x27`	`s2-s11`	saved	**callee**
`x28-x31`	`t3-t6`	temporary	caller

`docs/calling-convention.md` has all the details, with worked examples. Read it before you start this lab.

2 The rule

Caller-saved (`t0`–`t6`, `a0`–`a7`, `ra`): a function you call may destroy these. Need a value afterwards? Save it first.

Callee-saved (`s0`–`s11`, `sp`): a function you call must give these back unchanged. Want to use `s0` yourself? Save the old value and restore it before returning.

The second rule is important. The `s` registers are attractive because they survive calls — but they survive only because everyone restores them (and the same is expected from your code as well).

▶ *The tests check this directly. They put a known value in every `s` register, call your function, and look afterwards. A function that returns the right answer while clobbering `s0` is reported as `abi FAIL`: it would corrupt whatever its caller was in the middle of, and hence marks would be deducted.*

3 How these tests differ from Lab 1

Last week the tests ran your `main`. This week they ignore it. Your file is linked against a driver of ours which becomes `main`, calls your function with arguments of its choosing, and inspects what comes back.

This has two implications on your code. First, your function must be declared `.globl`, or the linking fails. The stubs already do this, so leave it UNCHANGED. Second, each case reports two independent results:

```
1-sum  int sum(int *arr, int n) -- a leaf function
  [value pass  abi ok]   case 0: n=6  arr=[4, 8, 15, 16, 23, 42]
  [value pass  abi FAIL] case 1: n=1  arr=[7]
         clobbered a callee-saved register (s0-s11) or left sp moved
```

`value` is whether you computed the right answer. `abi` is whether you honoured the convention. These two aspects will be scored separately.

Each stub also contains a `main` of its own that calls your function on the sample data. The tests ignore it; but it is used so that `make run` works properly (and therefore, will help you with your development).

4 Problems

Problem 1 — `1-sum.s: int sum(int *arr, int n)`

Add up the first `n` entries of the array and return the total. Return 0 when n is 0.

This function calls nothing, so it is a *leaf*: it needs no stack frame at all, as long as you stay in `t` and `a` registers. Start here — it is the one problem where you can ignore the stack entirely.

Problem 2 — `2-fact.s: int fact(int n)`

Return $n!$, computed **recursively**. Both 0! and 1! are 1. The tests go up to $12! = 479\,001\,600$, which is the largest that still fits in 32 bits.

This one calls itself, so two things must survive the call: `ra`, which `call` overwrites, and your copy of n, which you need afterwards to multiply by. That means a stack frame.

▶ *Make sure your base case really stops the recursion. If it does not, you will recurse until the stack grows down into your data, and the tests will report that your program ran past the instruction limit.*

Problem 3 — `3-matmul.s: int matmul(int *A, int *B, int *C)`

Multiply two 3×3 matrices, $C = A \times B$. Return 1 if every element of C is non-zero, and 0 otherwise.

Matrices are stored row by row, so element $[i][j]$ is the word at base $+ 4(3i + j)$. Overwrite C completely — do not assume it arrives full of zeros, because in one of the test cases it does not.

Three pointers and three nested loop counters is more than the `t` registers hold comfortably, so you will want `s` registers. You may use them. You must put them back.

5 Checking your work

```
$ make test                                  # all three problems
$ python3 tests/run_tests.py 2-fact          # just one
$ make run                                   # run your own main, for
    debugging
```

When something fails and you cannot see why, look at what actually executed:

```
$ cd programs
$ make runs/2-fact.iss
$ less runs/2-fact.iss
```

Watch `sp` and `ra` in that trace. A recursion bug is usually obvious once you can see `sp` marching down and never coming back up.

6 Submitting

1. Fill in `details.txt`: name, roll number, machine number.
2. Run `make clean`.
3. Leave everything else in place; the folder is collected at the end of the session.